

CORE JAVA 5TH SEM.

UNIT 1:

Java Architecture and features:

Java architecture is based on the principle of 'Write Once, Run Anywhere' (WORA), utilizing the Java Virtual Machine (JVM) to execute bytecode across different platforms. Key features include platform independence, object-oriented programming, automatic memory management, and a rich standard library.

Variables, Constant, Keywords, Data Types, Operators and Expression

1. Variables

Variables are used to store data that can be changed during the execution of a program. They have names and are assigned values.

2. Constants

Constants are similar to variables, but their values cannot be changed once assigned. In Java, constants are typically declared using the final keyword.

3. Keywords

Keywords are reserved words in a programming language that have special meaning. They cannot be used as identifiers (like variable names).

Common Java Keywords:

- `int`: Used to declare integer variables.
- `class`: Used to declare a class.
- `public`: An access modifier that allows visibility of classes or methods.

4. Data Types

Data types specify the type of data that a variable can hold. Java is a statically typed language, meaning the type of a variable must be declared at compile-time.

Primitive Data Types:

- `int`: Integer type (e.g., 1, 2, 3)
- `double`: Floating-point type (e.g., 1.5, 2.7)
- `char`: Character type (e.g., 'A', 'b')
- `boolean`: True or false values (e.g., true, false)

5. Operators

Operators are special symbols that perform operations on variables and values.

Types of Operators in Java:

- **Arithmetic Operators:** Used for mathematical operations.
 - +, -, *, /, %
- **Relational Operators:** Used to compare variables.
 - ==, !=, <, >, <=, >=
- **Logical Operators:** Used to combine conditional statements.
 - && (AND), || (OR), ! (NOT)

6. Expressions

An expression is a combination of variables, constants, operators, and method calls that evaluates to a single value.

Ex:

```
int result = (a + b) * 2;
```

Type Conversion

Type conversion refers to the process of converting a value from one data type to another. Java allows two types of type conversion:

1. **Implicit Type Conversion (Widening Conversion):**

This happens automatically when a smaller data type is converted to a larger data type. No data loss occurs in this process.

- **Explicit Type Conversion (Narrowing Conversion):**
This is done by the programmer to convert a larger data type to a smaller data type. This type of conversion may lead to data loss or exceptions.

Implicit Type Conversion Example:

```
int num = 10;  
  
double result = num;  
  
System.out.println(result);
```

Output is 10.0

Explicit Type Conversion Example:

```
double num = 10.5;  
  
int result = (int) num;  
  
System.out.println(result);
```

Output is 10

Type Casting

Type casting is a specific type of type conversion where one data type is explicitly converted into another, often using parentheses to denote the desired type. Type casting can be either widening or narrowing.

Example of Widening Type Casting:

When converting smaller data types to larger data types, the process is known as widening type casting. This is done automatically by Java.

Ex:

```
byte b = 10;
```

```
int i = b;
```

```
System.out.println(i);
```

Here byte is converted into int and output is 10

Example of Narrowing Type Casting:

Narrowing type casting occurs when you convert a larger data type to a smaller one. This requires explicit declaration to avoid data loss.

Ex:

```
float f = 9.78f;
```

```
int i = (int) f;
```

```
System.out.println(i);
```

Output is 9

Key Differences Between Type Conversion and Type Casting:

- **Type Conversion** can be implicit or explicit, while **Type Casting** is always explicit.

- **Type Conversion** often occurs automatically by the Java compiler, while **Type Casting** requires specific syntax by the programmer.

Decision Making Constructs.

1. if Statement

The if statement is used to test a condition. If the condition evaluates to true, the block of code inside the if statement is executed.

Syntax:

```
if (condition) {  
    // code to be executed if condition is true  
}
```

Example:

```
int age = 18;  
if (age >= 18) {  
    System.out.println("You are eligible to vote.");  
}
```

2. if-else Statement

The if-else statement provides an alternative path of execution. If the condition is false, the code in the else block is executed.

Syntax:

```
if (condition) {  
    // code to be executed if condition is true  
} else {  
    // code to be executed if condition is false  
}
```

Example:

```
int age = 16;  
if (age >= 18) {  
    System.out.println("You are eligible to vote.");  
} else {  
    System.out.println("You are not eligible to vote."); //  
Output: You are not eligible to vote.  
}
```

3. if-else if-else Ladder

This structure allows testing multiple conditions by chaining multiple if and else if statements.

Syntax:

```
if (condition1) {  
    // code to be executed if condition1 is true  
} else if (condition2) {  
    // code to be executed if condition2 is true
```

```
} else {  
    // code to be executed if neither condition is true  
}
```

Example:

```
int marks = 75;  
if (marks >= 90) {  
    System.out.println("Grade A");  
} else if (marks >= 75) {  
    System.out.println("Grade B"); // Output: Grade B  
} else {  
    System.out.println("Grade C");  
}
```

4. switch Statement

The switch statement is a multi-way branch statement that allows variable values to be tested for equality against a list of values. It's often used when you have multiple specific values to evaluate.

Syntax:

```
switch (expression) {  
    case value1:  
        // code to be executed if expression equals value1
```

```
    break;
case value2:
    // code to be executed if expression equals value2
    break;
default:
    // code to be executed if expression doesn't match
any case
}
```

Example:

```
int day = 3;
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday"); // Output:
Wednesday
        break;
```

default:

```
    System.out.println("Invalid day");  
}
```

Nesting Decision Making Constructs

Nesting refers to placing one control structure inside another. You can nest if statements, or use nested switch statements, among others.

Example of Nested if:

```
int age = 20;
```

```
boolean hasID = true;
```

```
if (age >= 18) {
```

```
    if (hasID) {
```

```
        System.out.println("You can enter the club."); //
```

Output: You can enter the club.

```
    } else {
```

```
        System.out.println("You need an ID to enter.");
```

```
    }
```

```
} else {
```

```
    System.out.println("You are not old enough to  
enter.");}
```

Example of Nested switch:

```
char grade = 'B';
switch (grade) {
    case 'A':
        System.out.println("Excellent!");
        break;
    case 'B':
        switch (grade) { // Nested switch
            case 'B':
                System.out.println("Good Job!"); // Output:
Good Job!
                break;
        }
        break;
    default:
        System.out.println("Keep Trying!");
}
```

Summary

- **Decision-making constructs** include if, if-else, and switch, which control the flow of the program based on conditions.

- **Nesting** allows you to place decision-making constructs within each other for more complex decision-making.

Java Methods.

1. Definition of Methods

A method in Java is a block of code that performs a specific task. It can take inputs, execute code, and optionally return a result. Methods help in code reusability, organization, and ease of maintenance.

Syntax:

```
returnType methodName(parameters) {  
    // method body  
    // code to be executed  
}
```

Example:

```
public int add(int a, int b) {  
    return a + b; // This method returns the sum of a  
    and b  
}
```

2. Scope of Methods

The scope of a method refers to where the method can be accessed from within the program. In Java, methods can be:

- **Instance Methods:** Methods that belong to an instance of a class. They can access instance variables and can be called on an object of the class.

Ex:

```
public class Calculator {  
    public int add(int a, int b) {  
        return a + b; // Instance method  
    }  
}
```

- **Static Methods:** Methods that belong to the class itself rather than any instance. They cannot access instance variables or instance methods directly.

Ex:

```
public class Calculator {  
    public static int multiply(int a, int b) {  
        return a * b; // Static method  
    }  
}
```

- **Access Modifiers:** The scope can also be controlled using access modifiers (public, private, protected). A private method can only be accessed within the same class.

3. Passing and Returning Arguments

Passing Arguments

When calling a method, you can pass arguments (input values) to it. Java supports two types of argument passing:

- **Pass by Value:** The value of the argument is passed to the method. Changes to the parameter inside the method do not affect the original argument.

Ex:

```
public void modifyValue(int number) {  
    number = 10; // Changes affect only the local copy  
}
```

- **Pass by Reference:** For objects, the reference to the object is passed. Changes to the object will affect the original object since they refer to the same memory space.

Ex:

```
public void modifyArray(int[] arr) {
```

```
arr[0] = 10; // This will modify the original array  
}
```

Returning Values

A method can return a value using the return statement. If a method's return type is not void, it must return a value of that type.

Ex:

```
public int square(int number) {  
    return number * number; // Returns the square  
    of the number  
}
```

4. Built-in Java Class Methods

Java has a rich set of built-in classes and methods that you can use. They are part of the Java Standard Library, which provides reusable code for various tasks.

Examples of Built-in Classes:

- **Math Class:** Contains methods for performing mathematical operations.
 - `Math.abs(x)`: Returns the absolute value of x.
 - `Math.max(a, b)`: Returns the maximum of two values.

Ex:

```
int max = Math.max(5, 10); // Output: 10
```

- **String Class:** Provides methods for manipulating strings.
 - `length()`: Returns the length of the string.
 - `substring(start, end)`: Returns a substring from the string.

Ex:

```
String str = "Hello";
```

```
int length = str.length(); // Output: 5
```

- **ArrayList Class:** A resizable array implementation of the List interface.
 - `add(element)`: Adds an element to the list.
 - `get(index)`: Retrieves the element at the specified index.

Ex:

```
ArrayList<String> list = new ArrayList<>();
```

```
list.add("Item1");
```

```
String item = list.get(0); // Output: Item1
```

UNIT 2:

Arrays:

What is an Array?

An **array** is a data structure that can store a fixed-size sequence of elements of the same type. Arrays are used to store collections of data, and they allow for efficient data management.

1. One-Dimensional Arrays (1D Arrays)

A **one-dimensional array** is like a list of elements. Here's how you can create and use a 1D array:

Creating a 1D Array

```
int[] array1D = {1, 2, 3, 4, 5};
```

Accessing Elements

You can access an element in the array using its index. Remember, indexing usually starts at 0.

```
// Accessing the second element
```

```
int secondElement = array1D[1]; // Returns 2
```

2. Multi-Dimensional Arrays

Multi-dimensional arrays are arrays of arrays. The most common type is the two-dimensional array, which can be thought of as a table (like a matrix).

Creating a 2D Array

Here's how to create a 2D array:

```
int[][] array2D = { {1, 2, 3},  
                    {4, 5, 6},  
                    {7, 8, 9} };
```

Accessing Elements

You can access elements in a 2D array using two indices, one for the row and one for the column:

```
// Accessing element in the second row and third  
column
```

```
int element = array2D[1][2]; // Returns 6
```

Dynamic Referencing of Arrays

Dynamic referencing means that you can allocate and access arrays while your program runs, instead of defining them statically (fixed size) at compile-time.

Dynamic Arrays in Java

In Java, you can use the `ArrayList` class which allows for dynamic arrays:

```
import java.util.ArrayList;  
  
ArrayList<Integer> dynamicArray = new ArrayList<>();  
dynamicArray.add(10); // Add an element
```

```
dynamicArray.add(20); // Add another element
```

Summary

- **1D Arrays:** A simple list of elements.
- **Multi-Dimensional Arrays:** Arrays of arrays (tables or matrices).
- **Dynamic Arrays:** Allow for the resizing of arrays during runtime using structures like lists in Python or ArrayList in Java.

Strings:

In Java, the String class is used to create and manipulate strings, which are sequences of characters. Here are the key features of the String class:

- **Immutable:** Once a String object is created, it cannot be altered. Any modifications create a new String object instead.
- **String Literals:** Strings can be created using double quotes.

1. Creating and Using String Objects

Creating Strings

You can create String objects in two main ways:

// Using string literals

```
String str1 = "Hello, World!";
```

```
// Using the new keyword
```

```
String str2 = new String("Hello, Java!");
```

Accessing String Methods

You can access various methods of the String class, which allow you to manipulate and retrieve information:

```
String str = "Java Programming";
```

```
// Length of the string
```

```
int length = str.length(); // Returns 17
```

```
// Accessing a character at a specific index
```

```
char firstChar = str.charAt(0); // Returns 'J'
```

2. Manipulating Strings

The String class provides several methods to manipulate strings:

- **Concatenation:** Combining two strings.

```
String str1 = "Hello";
```

```
String str2 = "World";
```

```
String combined = str1 + " " + str2; // "Hello World"
```

- **Substring:** Extracting a portion of the string.

```
String replaced = str.replace("Java", "Python"); //  
"Python Programming"
```

- **Replacing Characters:**

```
String replaced = str.replace("Java", "Python"); //  
"Python Programming"
```

- **Converting Case:**

```
String upper = str.toUpperCase(); // "JAVA  
PROGRAMMING"
```

```
String lower = str.toLowerCase(); // "java programming"
```

3. String Immutability and Equality

Immutability

Strings in Java are immutable. This means that once a String object is created, its value cannot be changed. For example:

```
String str = "Hello";
```

```
// Attempting to change str
```

```
str = str + " World"; // Creates a new String object
```

Equality

To check if two strings are equal, use the `.equals()` method, not the `==` operator.

The `==` operator checks for reference equality, while `.equals()` checks for value equality.

```
String str1 = "Hello";
```

```
String str2 = new String("Hello");
```

```
boolean areEqual = str1.equals(str2); // Returns true
```

```
boolean sameReference = (str1 == str2); // Returns false
```

4. Passing Strings to and from Methods

Strings can be passed to methods as parameters or returned as values. Here's an example:

```
public class Main {  
    // Method that takes a String as a parameter  
    public static void printMessage(String message) {  
        System.out.println(message);  
    }  
    // Method that returns a String  
    public static String greet(String name) {  
        return "Hello, " + name + "!";  
    }  
}
```

```
}  
  
public static void main(String[] args) {  
    printMessage("Welcome to Java!"); // Passing a  
string  
    String greeting = greet("Aditya"); // Returning  
string  
    System.out.println(greeting); // "Hello, Aditya!"  
}  
}
```

5. StringBuffer Class

The StringBuffer class is used to create mutable strings, which means that the content can be changed without creating a new object. This is useful for situations where you need to perform many modifications on a string.

Creating a StringBuffer

```
StringBuffer buffer = new StringBuffer("Hello");
```

Manipulating StringBuffer

You can perform various operations with StringBuffer:

- **Append:** Adding strings.

```
buffer.append(" World!"); // Now buffer contains "Hello  
World!"
```

- **Insert:** Inserting strings at a specific index.

```
buffer.insert(5, ","); // Now buffer contains "Hello,  
World!"
```

- **Replace:** Replacing a portion of the string.

```
buffer.replace(6, 11, "Java"); // "Hello, Java!"
```

- **Delete:** Deleting characters.

```
buffer.delete(5, 6); // "Hello Java!"
```

- **Convert to String:** You can convert a StringBuffer back to a String.

```
String result = buffer.toString(); // "Hello Java!"
```

I/O operation:

1. Simple I/O using System.out and Scanner Class

System.out

System.out is a standard output stream provided by Java. You can use it to print text to the console.

Example: Printing to Console

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!"); // Prints to  
        console with a newline  
    }  
}
```

```
System.out.print("Hello, "); // Prints without a  
newline
```

```
System.out.print("Java!"); // Continues on the  
same line
```

```
    }  
}
```

Scanner Class

The Scanner class is used to read input from various sources, including keyboard input (console). It's part of the java.util package.

Example: Reading Input from Console

```
import java.util.Scanner;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner = new Scanner(System.in);
```

```
        System.out.print("Enter your name: ");
```

```
        String name = scanner.nextLine();
```

```
        System.out.print("Enter your age: ");
```

```
        int age = scanner.nextInt();
```

```
        System.out.println("Hello, " + name + "! You are " +  
age + " years old.");
```

```
        scanner.close();
    }
}

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = scanner.nextLine();
        System.out.print("Enter your age: ");
        int age = scanner.nextInt(); // Read an integer
        System.out.println("Hello, " + name + "! You are " +
age + " years old.");
        scanner.close(); // Close the Scanner
    }
}
```

2. Byte and Character Streams

In Java, there are two types of streams for input and output:

- **Byte Streams:** Handle input and output of raw binary data (e.g., image files).

- **Character Streams:** Handle input and output of characters, and are suitable for text data.

Byte Streams

Byte streams are used to handle binary data. They are part of the java.io package and are represented by classes like InputStream and OutputStream.

Example: Reading and Writing Bytes

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
public class ByteStreamExample {
    public static void main(String[] args) {
        try (FileOutputStream fos = new
FileOutputStream("output.txt")) {
            String data = "Hello, Byte Stream!";
            fos.write(data.getBytes());
        } catch (IOException e) {
            e.printStackTrace();
        }
        try (FileInputStream fis = new
FileInputStream("output.txt")) {
```

```
int ch;
    while ((ch = fis.read()) != -1) {
        System.out.print((char) ch);
    }
} catch (IOException e) {
    e.printStackTrace();
}
}
```

Character Streams

Character streams are derived from Reader and Writer classes and are used to handle text data.

Example: Reading and Writing Characters

```
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
public class CharacterStreamExample {
    public static void main(String[] args) {
        try (FileWriter writer = new
FileWriter("output.txt")) {
```

```

        writer.write("Hello, Character Stream!");
    } catch (IOException e) {
        e.printStackTrace();
    }

    try (FileReader reader = new
FileReader("output.txt")) {
        int ch;
        while ((ch = reader.read()) != -1) {
            System.out.print((char) ch); // Read and print
characters
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

3. Reading and Writing from Console and Files

Reading from the Console

We have already covered how to read input from the console using the Scanner class. This is done via methods like `nextLine()`, `nextInt()`, etc.

Writing to the Console

You can write to the console using `System.out`, as shown previously.

Reading from Files

You can read from files using `FileReader` or `FileInputStream`, depending on whether you prefer character or byte processing:

```
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.IOException;
public class FileReadExample {
    public static void main(String[] args) {
        try (BufferedReader reader = new
BufferedReader(new FileReader("input.txt"))) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
```

```
        e.printStackTrace();
    }
}
}
```

Writing to Files

You can write to files using `FileWriter` or `FileOutputStream`. For character writing, `FileWriter` is often used:

```
import java.io.FileWriter;
import java.io.IOException;
public class FileWriteExample {
    public static void main(String[] args) {
        try (FileWriter writer = new
FileWriter("output.txt")) {
            writer.write("Writing to a file!"); // Write text to
a file
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

